

Aplicació dels Principis SÒLID a GestorXarxa

Principi de Responsabilitat Única (Single Responsibility Principle)

Algunes funcions de la classe GestorXarxa podrien ser separades en classes auxiliars o serveis diferents, com la gestió d'històric de modificacions o la lògica específica de càlcul de cabals.

Principi d'Obertura/Tancament (Open/Closed Principle)

A la classe GestorXarxa, es podrien introduir més abstraccions o usar patrons de disseny com Strategy o Command per les operacions de gestió permetria estendre el comportament sense modificar el codi existent.

Principi d'Inversió de Dependències (Dependency Inversion Principle)

Es podria refactoritzar el codi de GestorXarxa per dependre d'interfícies en lloc de classes concretes, ja que milloraria el desacoblament i facilitaria la substitució de components i testing.

Mal Disseny en GestorXarxa.java

- Mètodes recursius complexos: Alguns mètodes com tancarAixetesRec són recursius i podrien ser difícils de seguir o podrien causar problemes de rendiment o de pila si la xarxa és gran.
- Gestió d'estat global: La classe usa una variable “modificacions” per gestionar estats de modificacions anteriors. Això podria causar problemes de coherència si múltiples instàncies de GestorXarxa són emprades en paral·lel o en diferents contextos.
- Dependència en classes concretes: Similarment als altres fitxers, aquesta classe també fa ús de classes concretes en comptes d'interfícies, limitant la flexibilitat del codi i dificultant la substitució o extensió de components.
- Maneig de col·leccions sense seguretat de tipus: Els mètodes manipulen col·leccions d'aixetes de maneres que podrien causar NullPointerExceptions o errors en temps d'execució si les col·leccions estan buides o no inicialitzades correctament.

Aplicació dels Principis SOLID a XarxaAigua

Principi de Responsabilitat Única (Single Responsibility Principle)

El mètode `terminalAbonat(String dni)` podria ser refactoritzat per separar la lògica de cerca de la de comprovació, adherint-se millor a aquest principi.

Principi d'Obertura/Tancament (Open/Closed Principle)

Podriem refactoritzar el sistema per permetre afegir diferents tipus de nodes sense modificar els mètodes existents, potser mitjançant l'ús de polimorfisme o patrons de disseny.

Principi de Substitució de Liskov (Liskov Substitution Principle)

Aquest principi podria aplicar-se assegurant que les substitucions de `Terminal` i `Connexio` no trenquin la funcionalitat existent en la classe `XarxaAigua`.

Mal Disseny en `XarxaAigua.java`

- Duplicació de codi: Les funcions `afegir(Terminal nodeTerminal, ArrayList<Aixeta> llista)` i `afegir(Connexio nodeConnexio)` contenen codi molt similar per comprovar si l'element ja existeix a la xarxa i per afegir-lo. Això podria ser refactoritzat per reduir la duplicació i millorar la mantenibilitat.
- Maneig d'excepcions: El codi fa servir excepcions per controlar fluxos normals com la comprovació de nul·litat o la duplicació d'elements. Això pot no ser l'ideal ja que les excepcions haurien de reservar-se per a casos excepcionals, no per a fluxos de control esperats.
- Mètode llarg: El mètode `terminalAbonat(String dni)` és força llarg i fa diverses tasques: iteració sobre col·leccions, comprovació de tipus i cerca de condicions. Això podria ser simplificat i separar en mètodes més petits per millorar la claredat i reusabilitat.
- Ús de tipus concrets en lloc d'interfícies: Els paràmetres de mètodes com `afegir(Terminal nodeTerminal, ArrayList<Aixeta> llista)` fan servir `ArrayList` en lloc d'utilitzar `List` que és una interfície més genèrica. Això redueix la flexibilitat del codi.
- Acoblament fort: El codi sembla estar fortament acoblat, per exemple, amb les classes `Terminal` i `Connexio` directament utilitzades dins els mètodes. Això podria reduir la flexibilitat i la capacitat de testatge del codi.

Aplicació dels Principis SOLID al Main

Principi d'Inversió de Dependències (Dependency Inversion Principle)

Refactoritzar el codi per dependre d'abstraccions en lloc de classes concretes podria millorar l'extensibilitat i facilitar els tests. Per exemple, podríem injectar una interfície `XarxaAiguaInterface` en comptes d'usar directament la classe `XarxaAigua`.

Principi de Segregació d'Interfícies (Interface Segregation Principle)

Podríem crear interfícies més petites i específiques per a diferents funcionalitats d'usuari, permetent que els clients utilitzin només les parts que realment necessiten.

Principi d'Obertura/Tancament (Open/Closed Principle)

Si el codi es reestructura per utilitzar més abstraccions o patrons de disseny, permetria estendre les funcionalitats sense modificar el codi existent.

Mal Disseny en `Main.java`

- Acoblament fort: El codi té un acoblament fort amb la classe `XarxaAigua` i `GestorXarxa`, utilitzant molts mètodes específics d'aquestes classes. Això fa que el codi sigui menys flexible i més difícil d'adaptar o estendre.
- Responsabilitats mixtes: El codi en `Main.java` maneja la interacció d'usuari així com la lògica de negoci. Aquestes funcions podrien ser separades en classes diferents per manejar la lògica de negoci i la interacció d'usuari independentment.
- Maneig d'excepcions: No hi ha gestió explícita d'excepcions per errors d'entrada de l'usuari (com podrien ser formats de número incorrecte), el que pot conduir a crashes de l'aplicació o comportaments inesperats.
- Dependència en la classe `Scanner`: Totes les funcions depenen de `Scanner` per a la lectura de dades, el que limita la reusabilitat del codi en contexts on l'entrada no prové d'un flux estàndard de dades.
- Repetició de lògica de validació: Hi ha repetició en la comprovació de tipus d'objectes com es mostra en `if (a instanceof Origen)`. Aquest tipus de lògica podria ser encapsulat millor o gestionat a través de mètodes polimòrfics dins de les classes d'objectes.

Getters i Setters:

- A la classe Aixeta hi ha 2 setters, establirDemanda i establirCabalHipotetic. En comptes de fer mètodes per tal d'assignar atributs a un objecte de la classe es podria fer des del constructor.
- A la classe Aixeta també hi ha getters, aquests però amb el prefix obtenir, com per exemple obtenirDemanda. Crec que això no té una solució, ja que és la manera que tenim d'obtenir un atribut d'una classe

Aplicació dels Principis SÒLID a la interfície

Principi de Responsabilitat Única (Single Responsibility Principle)

La classe controladorInici té dues responsabilitats clares: gestionar l'obertura de fitxers i gestionar el pas a la següent finestra. En aquest cas, la responsabilitat no és única, però totes dues són necessàries per al bon funcionament; ja que són tasques específiques.

Principi d'Obertura/Tancament (Open/Closed Principle)

La classe controladorInici és oberta per l'extensió, ja que podem afegir més funcionalitats sense haver de modificar aquesta classe. Per exemple, podríem afegir un nou mètode per a altres accions relacionades amb la finestra d'inici sense haver de modificar els mètodes existents.

Principi d'Inversió de Dependències (Dependency Inversion Principle)

La classe controlador Inici està força acoblada a la llibreria JavaFX, pel fet que utilitza classes com FXMLLoader, Parent, Scene, etc. Podríem millorar aquesta dependència introduint una interfície que encapsuli l'accés a la finestra i als controls. Això faria que la classe no depengués directament de la implementació de JavaFX.

Resum General de tot el projecte, reflexió sobre decisions de Disseny

Avantatges:

Reusabilitat: En separar la lògica en diferents mètodes dins de la mateixa classe, es facilita la reutilització de codi en altres parts del sistema que necessitin realitzar funcions similars.

Fàcil manteniment: La clara separació i encapsulació de responsabilitats fa que sigui més fàcil mantenir i modificar el codi, ja que els canvis en un mètode són menys propensos a afectar altres parts de la classe.

Cohesió alta: Cada funció del programa està ben definida i executa una tasca específica, la qual cosa redueix la complexitat i fa que el manteniment sigui més senzill.

Fàcil d'ampliar: Afegir noves funcions o modificar les existents es facilita gràcies a la clara separació de funcionalitats.

Desavantatges:

Complexitat: Alguns mètodes semblen tenir molta lògica i múltiples responsabilitats, com tancarAixetes, que pot fer que el codi sigui més difícil d'entendre i mantenir a llarg termini.

Dependència en classes externes: El fort ús de les classes XarxaAigua i Aixeta pot fer que la classe GestorXarxa sigui menys flexible i més difícil d'utilitzar en un context diferent sense aquestes classes.

Dependència en entrada de consola: La dependència en Scanner per a la interacció amb l'usuari pot fer que el programa sigui menys adaptable a diferents entorns, com ara una interfície gràfica d'usuari o un entorn de tests automàtics.

Acoblament entre Main.java i altres classes: Tot i que hi ha una bona separació de responsabilitats, Main.java encara depèn directament de moltes classes i mètodes específics, el que podria dificultar la substitució o modificació de components específics sense afectar el codi principal.